Docker:
To create an image : **docker build -t system-health-dashboard . or can add a version of it calling 1.0.1**
To see all the images: **docker images**
To make a container and run : **docker run -d --name health-dashboard -p 5000:5000** and the image name
system-health-dashboard The first port is my PC port, and the second is the container port -d (detached to run in the background)
Docker ps : to see the current docker process container and using -a shows stopped too
docker run -d --name health-dashboard -p 5000:5000 system-health-dashboard:1.0.1

Kuberneetes: to manage and orchestrate containers. It monitors pods and can automatically replace them if they crash. You can set how many pod replicas you want running, for example (replicas:5) 5 pods across the cluster, and Kubernetes will try to keep that number running. It can also scale the number of pods up or down based on traffic or resource usage if autoscaling is configured, and it can distribute traffic between healthy pods to improve performance and reliability.

Local test of endpoints (py app.py)
Running on port 5000

```
PS C:\mthree\system-health-dashboard> py .\app.py
WARNING: This is a development server. Do not use it in a production
 deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.81:5000
```

Health success 200:

```
127.0.0.1 - - [24/Oct/2026 18:34:23] "GET /health HTTP/1.1" 200 -
```

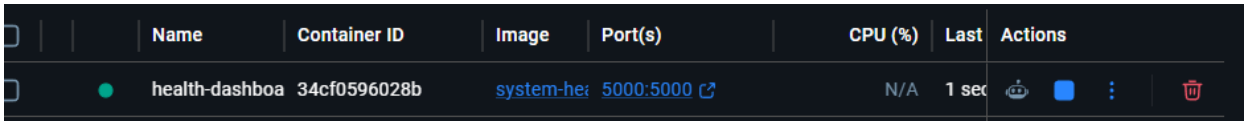
Environment success 200 :

```
127.0.0.1 - - [24/Oct/2026 18:34:46] "GET /environment HTTP/1.1" 200 -
```

Version success 200:

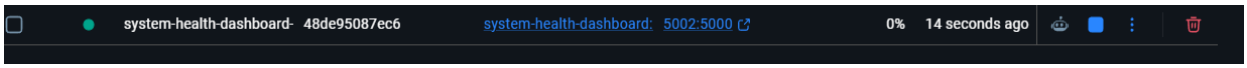
```
127.0.0.1 - - [24/Oct/2026 18:35:16] "GET /version HTTP/1.1" 200 -
```

Docker test:

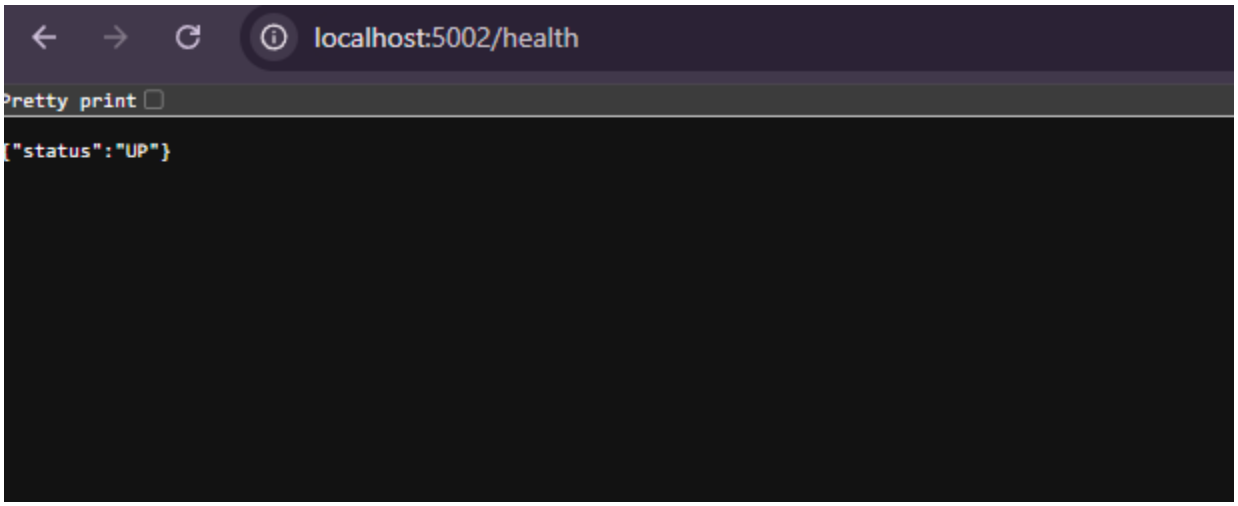


Build the docker image:
docker build -t system-health-dashboard:1.0.1

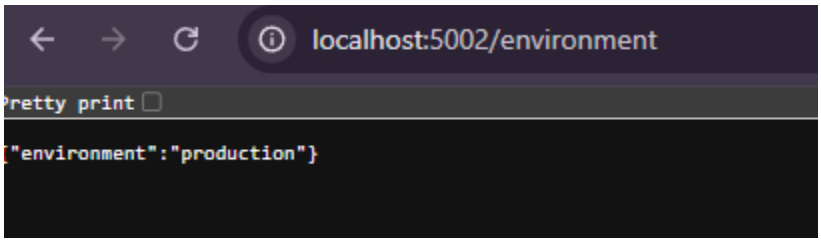
Start the container. Using port `5002` avoids conflicts with your existing container:
docker run -d --name system-health-dashboard-manual -p 5002:5000 -e ENVIRONMENT=production system-health-dashboard:1.0.1



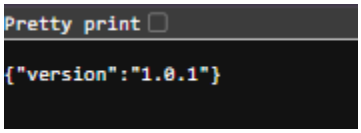
Test for health:



Environment:



Version:

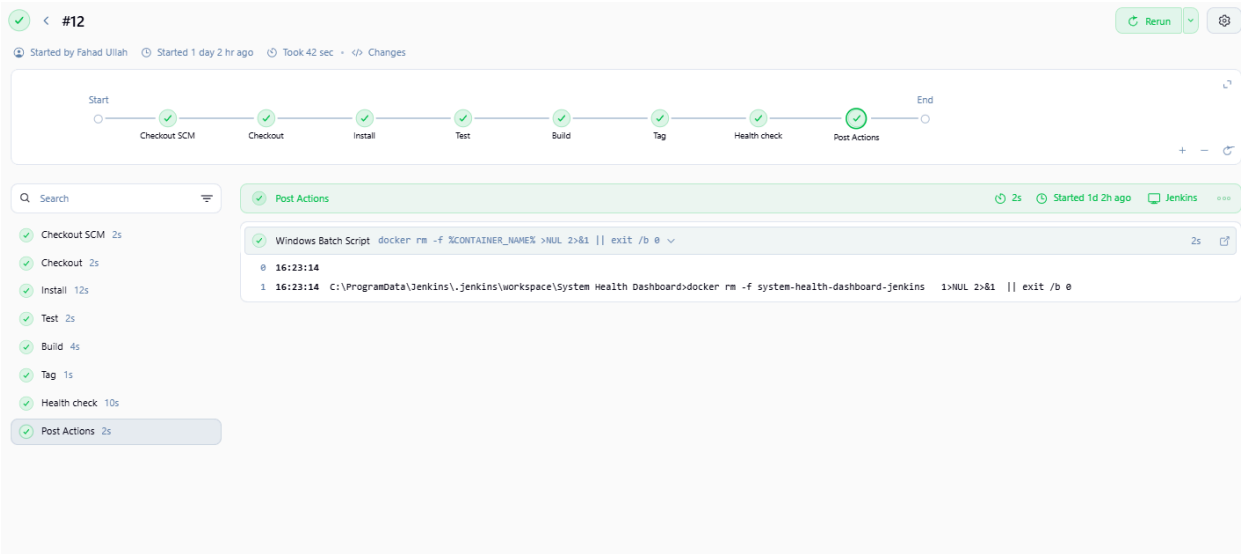


1.0.1 = patch/bug-fix release
1.1.0 = new backward-compatible feature
2.0.0 = major or breaking change

Jenkins: localhost:8080

Login -> create pipeline -> Use SCM and give link of the repo -> add */develop as thats the one we testing

Make Jenkinsfile -> add the rules commands




```
[Pipeline] End of Pipeline
Finished: SUCCESS
```

PyTest: py -m pytest
-m = run a Python module
pytest test-running module

```
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\mthree\system-health-dashboard
collected 4 items

tests\test_app.py ....                                                   [100%]

============================================================ 4 passed in 0.36s ============================================================
```

Merge Conflict:
A merge conflict occurred between `feature/conflict-version` and `feature/version-update` because both branches changed the `APP_VERSION` value in `app.py`. I resolved the conflict by keeping the intended version `1.0.1`, removing the conflict markers, and running the test suite to confirm that all tests passed.

Jenkins AI Assistance:

I used AI assistance to help understand, develop, and troubleshoot the Jenkins pipeline. The assistance included planning the pipeline stages, explaining Jenkins SCM configuration, and diagnosing Windows-specific errors involving Python paths, virtual

environments, Docker PATH configuration, Docker BuildKit, Docker Buildx, and the PowerShell health check.

The Jenkinsfile was developed and tested with the following stages:

- Checkout the repository
- Install Python dependencies
- Run pytest
- Build the Docker image
- Tag the image using the Jenkins build number
- Run the container and check the `/health` endpoint
- Remove the temporary container after the build

I reviewed and applied the suggested changes, tested the pipeline, and verified the endpoint and test results myself.

Dockerfile:
I also used AI assistance to create and understand the Dockerfile. The assistance helped me select the Python base image, install dependencies from `requirements.txt`, configure the application port, run the application as a non-privileged user, and use Docker BuildKit caching. I reviewed, tested, and verified the Dockerfile by building and running the container manually and through Jenkins.

Needs to be run on PowerShell, as it's a POST and not GET.

```
Invoke-RestMethod `
  -Uri http://localhost:5002/echo `
  -Method Post `
  -ContentType "application/json" `
  -Body '{"message":"Hello"}'
```

Which Git practice most improved the way you organised your work? Gitflow allowed the creation of branches for quick fixes by creating new branch of /features-fix, allowing for less conflict merging and manually reviews before merging. **pull requests** help by letting you review and verify changes before we eneter from develop to main, without interfering with the main branch

● **Where did automation detect or prevent an error?** When version app changed to 1.0.1, the test was checking for 1.0.0, so I had to go manually change the testing phase to check for the right version

● **What would you change before using this workflow in production?**
Before using this workflow in production,I would require approval before merging into main, and use Kubernetes to scale the application by creating additional pods when demand increases and distributing traffic between them.

● **Which step still depends on manual action, and how could it be automated?**
Merging to main branch, by adding a new pipeline script saying if all stages pass, it can merge to main, but if there is a conflict issue, it would have still have to be reviewed manually